

Siap, mari kita lanjutkan ke **Bab 2**. Ini adalah bagian paling krusial, karena struktur ini akan Anda gunakan di setiap game yang Anda buat ke depannya.

BAB 2: Struktur Dasar Program (Boilerplate)

Dalam pengembangan game, program tidak berjalan dari atas ke bawah lalu selesai. Game membutuhkan sebuah "**Game Loop**" (perulangan utama) yang berjalan terus menerus selama aplikasi terbuka.

Di dalam loop ini, program akan melakukan tiga hal utama secara berulang ribuan kali per detik:

1. **Event Handling:** Mengecek apakah pemain menekan tombol keyboard atau mouse.
2. **Update:** Memperbarui posisi karakter, skor, atau logika permainan.
3. **Draw:** Menggambar ulang semua objek ke layar agar terlihat seperti animasi.

Kerangka Dasar (Boilerplate)

Berikut adalah kode standar yang harus Anda pahami. Salin kode ini dan simpan sebagai `template.py`.

Python

```
import pygame
import sys

# 1. Inisialisasi Pygame
pygame.init()

# 2. Pengaturan Layar
LEBAR = 800
TINGGI = 600
screen = pygame.display.set_mode((LEBAR, TINGGI))
pygame.display.set_caption("Judul Game Saya")

# 3. Pengaturan Waktu (Clock)
clock = pygame.time.Clock()
FPS = 60 # Frame per second

# 4. Variabel Game
running = True

# --- GAME LOOP ---
while running:
    # A. Event Handling (Input dari pemain)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # B. Update (Logika game diletakkan di sini)
    # Contoh: posisi_karakter += kecepatan

    # C. Drawing (Menggambar ke layar)
    screen.fill((0, 0, 0)) # Membersihkan layar dengan warna hitam

    # ... (di sini nanti kita gambar objek)
```

```
# D. Refresh Layar (Menampilkan hasil gambar)
pygame.display.flip()

# E. Mengatur kecepatan game (FPS)
clock.tick(FPS)

# 5. Keluar dari Program
pygame.quit()
sys.exit()
```

Penjelasan Penting:

- **`pygame.time.Clock()` & `clock.tick(FPS)`:** Ini adalah kunci agar game berjalan dengan kecepatan yang sama di komputer mana pun. Jika kita tidak membatasi FPS (misalnya ke 60), game akan berjalan secepat kemampuan prosesor komputer Anda (bisa ratusan/ribuan FPS), yang membuat game menjadi terlalu cepat dan tidak stabil.
 - **`screen.fill((0, 0, 0))`:** Bayangkan ini sebagai menghapus papan tulis sebelum menulis materi baru. Di setiap *frame*, kita harus "menghapus" gambar lama agar tidak meninggalkan bekas *trail* (jejak) yang berantakan.
 - **`pygame.display.flip()`:** Pygame menggunakan teknik *Double Buffering*. Kita menggambar semua objek di "layar tersembunyi" terlebih dahulu, baru kemudian menampilkannya secara instan ke layar asli agar tidak terjadi *flickering* (layar berkedip).
-

Latihan Kecil

Coba ubah warna latar belakangnya. Ganti `(0, 0, 0)` di dalam `screen.fill()` dengan angka lain (Rentang 0-255 untuk format RGB).

- Contoh: `(255, 0, 0)` akan membuat latar belakang menjadi merah.